1. Сначала пойми весь маршрут

RateRadar KZ • разбор готового проекта за один учебный час

Твоё приложение показывает обменники и демонстрационные курсы USD. Пользователь выбирает город, открывает обменник и отмечает его сердцем. Интернет не нужен: 12 обменников записаны в SampleData.kt. Есть три экрана, но одна Android Activity.

Это схема связей, не Kotlin-код

```
Запуск Android
  MainActivity -> RateRadarTheme -> RateRadarNavGraph
                                      |
                ExchangeOfficeListScreen
                   | OfficeCard -> RateBlock
                   | нажатие на карточку
                OfficeDetailsScreen
                   | добавить сердце
                FavoritesScreen -> OfficeCard

Данные: SampleData. Общее избранное: RateRadarNavGraph.
```

Ключевая мысль: данные и состояние определяют интерфейс. Функции Compose описывают, что показывать. Нажатие меняет состояние или маршрут, после чего Compose обновляет нужные части интерфейса.

Как потратить именно час

0-6 минут: страницы 1-3, чтобы ориентироваться в файлах и записи кода. 6-12: страницы 4-6, запуск, навигация и данные. 12-26: страницы 7-10, устройство списка и карточки. 26-42: страницы 11-14, избранное, переходы и детали. 42-50: страницы 15-16, оформление и сборка. 50-60: вопросы и две правки на страницах 17-18.

> **Открой исходный проект RateRadarKZ в Android Studio. Читай файл по указанному пути и находи выделенный фрагмент. Учебный android-practice сейчас не нужен: защита будет по готовому проекту.**

Задача часа - уметь проследить действия и найти место для изменения. Знать наизусть все импорты и цвета не требуется. Ответить на совершенно любой вопрос за час нельзя обещать; понимание цепочки полезнее заученных определений.

2. Где что лежит

Корень: /Users/alishermutaliyev/Projects/kbtu_android

```
kbtu_android/
  app/
    build.gradle.kts              настройки приложения
    src/main/
      AndroidManifest.xml       сведения для Android
      java/kz/edu/kbtu/rateradar/
        MainActivity.kt
        model/ExchangeOffice.kt
        data/SampleData.kt
        navigation/RateRadarNavGraph.kt
        ui/screens/             три экрана
        ui/components/          три повторяемых элемента
        ui/theme/               оформление и размеры
      res/                      строки, рисунки, XML
    src/test/                   обычные тесты
  build.gradle.kts              версии плагинов
  settings.gradle.kts           модули и источники библиотек
  gradlew                       запуск сборки
  design/                       эскизы
  sis2/                         задачи Kotlin, не UI приложения
```

Android Studio в режиме Android группирует файлы удобнее, чем они реально лежат: manifests, kotlin+java и res. Папка java содержит и Kotlin. app - модуль, из которого собирается устанавливаемое приложение. build/ и .gradle/ содержат результаты и служебные данные, их обычно не редактируют.

ui/screens: ExchangeOfficeListScreen - список; OfficeDetailsScreen - детали; FavoritesScreen - избранное. ui/components: OfficeCard - карточка; RateBlock - один курс; EmptyState - сообщение вместо пустого содержимого.

ui/theme: Color.kt - конкретные цвета; Theme.kt - светлая/тёмная схема; Type.kt - стили текста; Dimens.kt - общие размеры. model хранит описание записи, data - сами записи. Папка domain, если видна пустой, ничего не выполняет: само наличие папки не создаёт логику.

> **package kz.edu.kbtu.rateradar.ui.components - адрес объявлений файла. import позволяет использовать чужое имя коротко. Импорт не запускает функцию. R и BuildConfig создаются инструментами сборки; писать их вручную не нужно.**

3. Как читать Kotlin без ступора

Эта страница - ключ к фрагментам дальше; не отдельный курс языка

Сокращённая сигнатура OfficeCard.kt; тело заменено комментарием

```
fun OfficeCard(
    office: ExchangeOffice,
    isFavorite: Boolean,
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
) { /* тело функции */ }
```

fun объявляет функцию. В круглых скобках - её входные параметры. office: ExchangeOffice читается «параметр office имеет тип ExchangeOffice». Boolean - true или false. modifier получает значение по умолчанию, если вызывающий код его не передал. Фигурные скобки ограничивают тело функции.

() -> Unit - тип функции без входных параметров и без полезного возвращаемого значения. (Int) -> Unit - тип функции, которая принимает целое число. Это позволяет передавать действия как значения: экран получает инструкцию, что выполнить после нажатия.

Реальная запись внутри вызова OfficeCard в экране списка

```
onClick = { onOfficeClick(office.id) }
```

Слева onClick - имя параметра карточки. Справа { ... } - лямбда, то есть функция без отдельного имени. Она вызовет onOfficeClick с id конкретного обменника, когда пользователь нажмёт карточку. Передать действие и выполнить его сейчас - разные операции.

val - переменной нельзя присвоить другое значение; var - можно. = присваивает или задаёт аргумент; == сравнивает. Точка в office.id читает свойство объекта. Типы: Int - целое, Double - дробное, String - текст. Set<Int> - множество уникальных целых; List<ExchangeOffice> - упорядоченный список записей.

{ id -> ... } явно называет параметр лямбды. Если параметр один и имя опущено, он называется it: filter { it.city == selectedCity }. Стрелка отделяет параметр от действий. Последняя запятая разрешена. Запись f(a = 1) передаёт именованный аргумент.

> **@Composable - метка функции, участвующей в построении интерфейса Compose. Это не экран само по себе: функцию ещё должен вызвать другой код. Text(...), Row(...) и наши OfficeCard(...) - вызовы таких функций.**

4. С чего приложение запускается

MainActivity.kt → ui/theme/Theme.kt → navigation/RateRadarNavGraph.kt

MainActivity.kt | строки 10-20

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            RateRadarTheme {
                RateRadarNavGraph()
            }
        }
    }
}
```

MainActivity наследуется от ComponentActivity: двоеточие указывает родительский класс, () вызывает его конструктор. Activity - компонент Android с собственным жизненным циклом; в нашем проекте она служит контейнером для трёх экранов Compose.

Android вызывает onCreate при создании Activity. override означает «переопределяю метод родителя». super.onCreate выполняет родительскую часть инициализации. savedInstanceState: Bundle? - контейнер ранее сохранённых значений или null; ? разрешает отсутствие значения. Мы передаём его родителю.

enableEdgeToEdge разрешает интерфейсу занимать область до краёв окна, включая пространство за системными панелями. Поэтому на экранах нужно учитывать отступы. setContent подключает Compose и задаёт содержимое окна. Внутри обычная вложенность вызовов: тема оборачивает граф навигации.

Что делает оболочка темы

RateRadarTheme не открывает экран и не загружает данные. Она выбирает цвета и стили и делает их доступными вложенному интерфейсу через MaterialTheme. RateRadarNavGraph уже решает, какой экран показывать. Подробные цвета разберём на странице 15.

Почему обновление UI не запускает приложение заново

Compose может повторно выполнять функции интерфейса, когда меняются прочитанные ими наблюдаемые значения. Это recomposition - пересчёт описания интерфейса. Это не повторный onCreate и не обязательная перерисовка всего экрана. Поэтому в теле UI не нужно вручную запускать переход на другой экран.

> **Ответ на защите: «Android создаёт MainActivity. В onCreate я подключаю Compose через setContent, применяю RateRadarTheme и вызываю граф с тремя экранами».**

5. Кто выбирает первый экран

navigation/RateRadarNavGraph.kt

navigation/RateRadarNavGraph.kt | строки 17-23

```
object Routes {
    const val OFFICES = "offices"
    const val DETAILS = "details/{officeId}"
    const val FAVORITES = "favorites"

    fun details(officeId: Int) = "details/$officeId"
}
```

Routes - один объект с адресами экранов. const val - неизменная константа, известная при компиляции. OFFICES и FAVORITES - готовые адреса. DETAILS - шаблон: фигурные скобки внутри строки обозначают место для аргумента officeId.

details(3) возвращает строку details/3: $officeId подставляет число. Это адрес внутри приложения, не интернет-ссылка. После = в объявлении этой функции сразу записано возвращаемое выражение.

navigation/RateRadarNavGraph.kt | строки 27-27

```
val navController = rememberNavController()
```

navigation/RateRadarNavGraph.kt | строки 39-48

```
NavHost(navController = navController, startDestination = Routes.OFFICES) {

    composable(Routes.OFFICES) {
        ExchangeOfficeListScreen(
            favoriteIds = favoriteIds,
            onOfficeClick = { id -> navController.navigate(Routes.details(id)) },
            onFavoriteClick = { id -> toggleFavorite(id) },
            onOpenFavorites = { navController.navigate(Routes.FAVORITES) },
        )
    }
```

NavController управляет текущим маршрутом и историей переходов. rememberNavController удерживает контроллер между пересчётами интерфейса. NavHost - место, где отображается текущий экран. startDestination = Routes.OFFICES выбирает первый маршрут.

composable(Routes.OFFICES) регистрирует адрес и код его экрана. Не путай: composable(...) - функция библиотеки навигации; @Composable - аннотация над функцией UI. Граф знает, как переходить; экран списка получает готовые действия.

favoriteIds = favoriteIds: слева имя параметра экрана, справа общее значение графа. onOfficeClick получает лямбду: принять id и открыть details/id. onOpenFavorites получает лямбду без аргументов: открыть favorites. onFavoriteClick передаёт id в toggleFavorite - его разберём на странице 11.

Граф связывает экраны; сами экраны рисуют UI и сообщают о событиях через callbacks.

6. Откуда берутся обменники

model/ExchangeOffice.kt → data/SampleData.kt → экран списка

model/ExchangeOffice.kt | строки 3-12

```
data class ExchangeOffice(
    val id: Int,
    val name: String,
    val city: String,
    val address: String,
    val workingHours: String,
    val usdBuy: Double,
    val usdSell: Double,
    val updatedAt: String,
)
```

Это описание одного обменника. data class хранит запись и автоматически получает сравнение по содержимому, toString и copy. val перед полем позволяет читать его как свойство, но запрещает перезаписывать поле. id - постоянный идентификатор, не номер текущей строки на экране.

SampleData.kt: фрагмент с переносами; остальные записи опущены

```
object SampleData {
    const val ALL_CITIES = "All"
    val cities = listOf(ALL_CITIES, "Almaty", "Astana",
        "Shymkent", "Karaganda")

    val offices = listOf(
        ExchangeOffice(1, "Abay Exchange", "Almaty",
            "Abay Ave 52", "09:00 - 20:00",
            512.50, 515.00, "10:30"),
        // дальше ещё 11 записей
    )
}
```

object создаёт один общий объект SampleData. Не нужно писать SampleData(). listOf создаёт список. Каждый ExchangeOffice(...) создаёт одну запись; здесь аргументы позиционные и идут в том же порядке, что поля модели. Значит 512.50 - usdBuy, а 515.00 - usdSell.

В cities перечислены варианты фильтра. All - специальный вариант «показать все», это не город. В offices: 5 записей Almaty, 4 Astana, 3 Shymkent. В Karaganda записей нет специально, чтобы показать пустое состояние.

usdBuy - курс, по которому обменник покупает USD; usdSell - продаёт. updatedAt - записанный текст времени, он не обновляется таймером. Адрес и часы используются на Details, хотя в карточке не показываются. Данные не приходят с сервера и не хранятся в базе.

> **Поменяешь имя или курс в SampleData - после запуска их получат и карточка, и Details: оба используют одни записи. filter создаёт отобранный список, исходные 12 записей не удаляет.**

7. Scaffold: что куда ставится

ui/screens/ExchangeOfficeListScreen.kt | сначала каркас, затем содержимое

ui/screens/ExchangeOfficeListScreen.kt | строки 54-69

```
Scaffold(
    topBar = {
        TopAppBar(
            title = { Text(stringResource(R.string.title_offices)) },
            actions = {
                IconButton(onClick = onOpenFavorites) {
                    Icon(
                        imageVector = Icons.Filled.Favorite,
                        contentDescription = stringResource(R.string.open_favorites),
                    )
                }
            },
        )
    },
) { innerPadding ->
    Column(modifier = Modifier.padding(innerPadding)) {
```

Scaffold - готовый каркас экрана Material 3. У него есть места для верхней панели, нижней панели, плавающей кнопки и основного содержимого. Здесь используем topBar и содержимое. Сам Scaffold не создаёт список и не занимается переходами.

topBar = { ... } передаёт функцию, описывающую верхнюю панель. TopAppBar строит её. title = { Text(...) } описывает заголовок. actions = { ... } описывает кнопки справа. Это разные блоки содержимого, хотя все записаны фигурными скобками.

IconButton(onClick = onOpenFavorites) получает готовое действие для нажатия. Следующий блок { Icon(...) } задаёт, что видно внутри кнопки. Он не выполняется «после нажатия»: это описание значка. imageVector выбирает изображение сердца, contentDescription даёт текстовое описание для доступности.

Блок после Scaffold получает innerPadding - отступы, рассчитанные с учётом панелей. Стрелка -> отделяет входной параметр от тела лямбды. Имя innerPadding придумал автор; можно назвать иначе, смысл не изменится.

Scaffold передаёт отступы, но не применяет их за нас к содержимому. Column(modifier = Modifier.padding(innerPadding)) применяет их. Без этого содержимое может попасть под верхнюю или системную панель. Column дальше располагает фильтры и список вертикально.

> **stringResource(R.string.title_offices) читает текст из res/values/strings.xml. R.string - сгенерированный адрес ресурса. Поэтому заголовок меняется в XML, а не поиском текста по всем Kotlin-файлам.**

8. Нажал город - откуда новый список

ui/screens/ExchangeOfficeListScreen.kt | состояние фильтра

ui/screens/ExchangeOfficeListScreen.kt | строки 46-52

```
var selectedCity by rememberSaveable { mutableStateOf(SampleData.ALL_CITIES) }

val offices = if (selectedCity == SampleData.ALL_CITIES) {
    SampleData.offices
} else {
    SampleData.offices.filter { it.city == selectedCity }
}
```

selectedCity хранит выбранный город. mutableStateOf создаёт наблюдаемое значение: Compose замечает его изменение. rememberSaveable удерживает его между пересчётами и может восстановить при пересоздании Activity. by позволяет обращаться сразу к значению, без записи .value.

Обычная var сама по себе не сообщает Compose о присваивании. Здесь важна вся конструкция, а не только var. Если каждый пересчёт создавать новое состояние без remember/rememberSaveable, можно снова получить начальное All.

if возвращает список выбранной ветки. Для All берём все данные. Иначе filter проверяет каждый обменник: it - текущая запись; it.city == selectedCity возвращает true или false. Только записи с true попадают в новый список offices. Переменная offices - val, но при новом выполнении функции она вычисляется заново.

ui/screens/ExchangeOfficeListScreen.kt | строки 75-81

```
items(SampleData.cities) { city ->
    FilterChip(
        selected = city == selectedCity,
        onClick = { selectedCity = city },
        label = { Text(city) },
    )
}
```

LazyRow - горизонтальный прокручиваемый список городов. items перебирает города, и city - текущая строка. FilterChip - готовая кнопка выбора фильтра. selected определяет её активный вид. onClick меняет выбранный город. label задаёт текст внутри.

Порядок событий; схема, не фрагмент файла

```
Нажал Astana -> selectedCity = "Astana"
  -> Compose пересчитывает читающий его UI
  -> filter оставляет id 6, 7, 8, 9
  -> активен chip Astana; в списке четыре карточки
```

> **Для Karaganda filter вернёт пустой список. Следующий if (offices.isEmpty()) покажет EmptyState. Меняется отображение, а не содержимое SampleData.**

9. Как из списка получаются карточки

ui/screens/ExchangeOfficeListScreen.kt → ui/components/OfficeCard.kt

ui/screens/ExchangeOfficeListScreen.kt | строки 92-102

```
LazyColumn(
    contentPadding = PaddingValues(Spacing.large),
    verticalArrangement = Arrangement.spacedBy(Spacing.medium),
) {
    items(offices, key = { it.id }) { office ->
        OfficeCard(
            office = office,
            isFavorite = office.id in favoriteIds,
            onClick = { onOfficeClick(office.id) },
            onFavoriteClick = { onFavoriteClick(office.id) },
            modifier = Modifier.fillMaxWidth(),
```

LazyColumn - вертикальный список со встроенной прокруткой. Он создаёт и размещает элементы по мере необходимости. Column просто располагает детей вертикально; прокрутку к ней нужно подключать отдельно. Для многих однотипных карточек используем LazyColumn.

contentPadding задаёт отступы по краям содержимого списка. Arrangement.spacedBy задаёт промежутки между соседними карточками. Здесь Spacing.large = 16.dp, Spacing.medium = 8.dp. Это не один и тот же отступ.

items(offices, ...) добавляет элементы из отфильтрованного списка. key = { it.id } даёт каждой карточке постоянный уникальный ключ. При изменении порядка или фильтра Compose может узнавать элемент по id, а не только по позиции. Ключ не является обработчиком клика и не делает навигацию.

После items идёт лямбда { office -> ... }: она получает текущую запись и описывает её карточку. office = office передаёт объект. isFavorite = office.id in favoriteIds проверяет, есть ли id в общем наборе; результат - Boolean.

onClick = { onOfficeClick(office.id) } связывает нажатие именно с этим обменником. onFavoriteClick похож, но отвечает за сердце. Карточка получает функции без аргументов: id уже подставлен в лямбдах родительского экрана. Куда уйдёт событие дальше, решает граф.

Что делает modifier

Modifier - цепочка настроек размера, отступов, поведения и рисования элемента. fillMaxWidth занимает доступную ширину. modifier с маленькой буквы - входной параметр нашей функции; Modifier с большой - тип и начальная пустая цепочка. Переданный modifier позволяет вызывающему экрану настроить карточку снаружи.

> **Список рисует OfficeCard много раз с разными данными. В Favorites используется тот же компонент. Меняя OfficeCard, меняешь карточки сразу на обоих экранах.**

10. Что внутри одной карточки

ui/components/OfficeCard.kt → ui/components/RateBlock.kt

Структура реального OfficeCard; схема, не Kotlin-код

```
Card: onClick, modifier, цвета
  Row: padding 16.dp, CenterVertically
    Column: weight(1f)
      Text: имя, titleMedium, одна строка, многоточие
      Text: город и время, bodySmall
      Row: два RateBlock рядом, промежуток 8.dp
    IconButton: сердце; отдельный onFavoriteClick
```

Row располагает детей по горизонтали: слева информация, справа сердце. Column внутри ставит имя, город и курсы друг под другом. weight(1f) отдаёт информационной колонке оставшуюся ширину после измерения кнопки сердца. f означает число Float; это доля, а не пиксели.

verticalAlignment = Alignment.CenterVertically выравнивает детей Row по вертикали. horizontalArrangement задаёт распределение по горизонтали. У Column направления меняются местами: verticalArrangement распределяет детей сверху вниз, horizontalAlignment выравнивает поперёк.

ui/components/OfficeCard.kt | строки 48-52

```
text = office.name,
style = MaterialTheme.typography.titleMedium,
color = MaterialTheme.colorScheme.onSurface,
maxLines = 1,
overflow = TextOverflow.Ellipsis,
```

style задаёт набор параметров текста из темы. color - роль цвета. maxLines = 1 ограничивает строкой; Ellipsis показывает многоточие при нехватке места. Само имя в данных остаётся полным. На Details ограничение maxLines отсутствует, поэтому длинное имя переносится.

CardDefaults.cardColors настраивает цвета Card. if (isFavorite) выбирает заполненное сердце, иначе FavoriteBorder. RateBlock получает label и rate. Surface задаёт поверхность: цвет фона и форму. Внутри Column рисует подпись Buy USD/Sell USD и число. onSecondaryContainer - цвет текста на secondaryContainer; приставка on здесь не означает событие.

Второй Text читает stringResource(R.string.rate_value, rate). В XML шаблон %1$.2f ₸: первый аргумент - дробное число, два знака после разделителя. Поэтому 512.5 отображается с двумя дробными знаками; сам Double не меняется. В Details два RateBlock имеют weight(1f), поэтому делят доступную ширину поровну.

> **Порядок Modifier важен: операции оборачивают элемент последовательно. Например, clip(...).padding(...) и padding(...).clip(...) обрезают разные области. padding внутри Card даёт место от её края до содержимого; spacedBy даёт место между детьми.**

11. Почему сердце меняется везде

navigation/RateRadarNavGraph.kt | общее состояние и события

navigation/RateRadarNavGraph.kt | строки 29-37

```
var favoriteIds by remember { mutableStateOf(setOf<Int>()) }

fun toggleFavorite(officeId: Int) {
    favoriteIds = if (officeId in favoriteIds) {
        favoriteIds - officeId
    } else {
        favoriteIds + officeId
    }
}
```

favoriteIds - множество id избранных обменников. setOf<Int>() создаёт пустой набор целых чисел. mutableStateOf оборачивает его в наблюдаемое состояние. remember сохраняет эту обёртку между пересчётами графа. by даёт короткую запись чтения и присваивания; getValue/setValue нужны для неё в импортах.

toggleFavorite принимает id. in проверяет наличие. Если id есть, favoriteIds - officeId создаёт новый набор без него; иначе + добавляет его. if возвращает набор выбранной ветки, а присваивание сохраняет его в состоянии. Набор заменяется, а не мутируется скрыто внутри.

Полная цепочка одного нажатия

Эти части находятся в трёх разных файлах

```
OfficeCard: IconButton(onClick = onFavoriteClick)
  -> экран: { onFavoriteClick(office.id) }
  -> граф: { id -> toggleFavorite(id) }
  -> favoriteIds = новый набор
  -> новый isFavorite у карточки и Details
  -> новый отбор записей на Favorites
```

Callback - функция, которую передали для реакции на событие. Карточка не хранит отдельное сердце внутри себя. Она получает isFavorite и вызывает callback. Общее состояние поднято в родителя всех трёх экранов: это state hoisting, буквально перенос владения состоянием выше.

Почему не хранить состояние в каждой карточке? Тогда у карточки в списке, карточки Favorites и кнопки Details были бы независимые версии. Здесь один набор - один источник значения. В обработчике нажатия меняем набор; при построении UI читаем его и выбираем нужный вид.

Что сохранится при перезапуске

remember переживает recomposition, но не пересоздание Activity или обычный новый запуск. Внутри живого NavGraph переходы сохраняют избранное. На диск оно не записывается. rememberSaveable у города умеет восстановление через saved state при пересоздании, но тоже не заменяет постоянное хранилище.

> **Пример: {} → нажал id 3 → {3} → нажал id 6 → {3,6} → снова id 3 → {6}. И список, и Details получают ответ из этого одного набора.**

12. Нажал карточку - открыл Details

navigation/RateRadarNavGraph.kt → ui/screens/OfficeDetailsScreen.kt

navigation/RateRadarNavGraph.kt | строки 51-60

```
    route = Routes.DETAILS,
    arguments = listOf(navArgument("officeId") { type = NavType.IntType }),
) { backStackEntry ->
    val officeId = backStackEntry.arguments?.getInt("officeId") ?: 0
    OfficeDetailsScreen(
        officeId = officeId,
        isFavorite = officeId in favoriteIds,
        onBackClick = { navController.popBackStack() },
        onFavoriteClick = { id -> toggleFavorite(id) },
    )
```

Карточка id 3 в итоге вызывает navigate(Routes.details(3)), то есть navigate("details/3"). Граф сопоставляет адрес с шаблоном details/{officeId}. navArgument задаёт имя аргумента и его тип: NavType.IntType - целое число.

backStackEntry - запись этого посещения экрана в истории. arguments хранит его аргументы. ?.getInt(...) выполняет чтение только если arguments не null; иначе результат null. ?: 0 выбирает запасной 0 при null. getInt для отсутствующего ключа также может вернуть 0. В данных id 0 нет, поэтому он приведёт к сообщению Office not found.

officeId передаётся в Details. isFavorite = officeId in favoriteIds вычисляет состояние кнопки. onBackClick получает функцию возврата. onFavoriteClick снова связан с общим toggleFavorite.

ui/screens/OfficeDetailsScreen.kt | строки 52-52

```
val office = SampleData.offices.find { it.id == officeId }
```

find перебирает записи и возвращает первую с совпавшим id либо null. Это поиск по идентификатору, не обращение offices[officeId]. После фильтра позиции меняются, а id остаётся тем же. В Details проверяем office == null: пустое сообщение, иначе содержимое. В else Kotlin знает, что office не null; это smart cast.

Как работает возврат

Стек посещений: последнее снимается первым

```
[offices]
  navigate("favorites") -> [offices, favorites]
  navigate("details/3") -> [offices, favorites, details/3]
  popBackStack()        -> [offices, favorites]
```

Back не угадывает нужный экран, а снимает верхнюю запись. Поэтому Details, открытый через Favorites, возвращает в Favorites. navigate("offices") сделал бы новое посещение списка вместо возврата. Системный Back тоже работает с историей навигации; собственная стрелка вызывает наш callback с popBackStack.

Имя officeId должно совпасть в шаблоне, navArgument и getInt. Через маршрут передаём id, а объект находим в данных.

13. Что рисует экран деталей

ui/screens/OfficeDetailsScreen.kt | одна запись, обычная прокрутка

ui/screens/OfficeDetailsScreen.kt | строки 79-84

```
Column(
    modifier = Modifier
        .padding(innerPadding)
        .verticalScroll(rememberScrollState())
        .padding(Spacing.large),
    verticalArrangement = Arrangement.spacedBy(Spacing.large),
```

Details получает одну найденную запись. Здесь не список однотипных строк, а небольшой набор разных блоков, поэтому Column с verticalScroll. rememberScrollState хранит позицию прокрутки между пересчётами; verticalScroll подключает её к колонке. Это не состояние избранного.

Сначала padding(innerPadding) учитывает панели Scaffold. Далее подключаем прокрутку и добавляем padding(Spacing.large), то есть 16.dp вокруг внутреннего содержимого. spacedBy(Spacing.large) делает промежутки между блоками.

ui/screens/OfficeDetailsScreen.kt | строки 86-93

```
Image(
    painter = painterResource(R.drawable.exchange_placeholder),
    contentDescription = stringResource(R.string.office_image_description, office.name),
    modifier = Modifier
        .fillMaxWidth()
        .height(Sizes.detailsImageHeight)
        .clip(MaterialTheme.shapes.medium),
    contentScale = ContentScale.Crop,
```

painterResource загружает локальный рисунок из res/drawable/exchange_placeholder.xml. Это векторная иллюстрация, не реальное фото обменника. contentDescription - описание для программы чтения экрана; описание исправлено на Illustration, чтобы соответствовать рисунку.

fillMaxWidth растягивает область изображения, height задаёт 180.dp, clip обрезает по форме medium. ContentScale.Crop масштабирует рисунок с сохранением пропорций до заполнения области; лишнее может обрезаться.

Дальше Text показывает полное имя с headlineSmall. Card содержит Column с городом, адресом и часами. Row ставит два RateBlock рядом, оба weight(1f). Button Add и OutlinedButton Remove - две ветки if (isFavorite). Нажатие любой вызывает onFavoriteClick(office.id), а не меняет локальную переменную.

buttonModifier задаёт fillMaxWidth и heightIn(min = Sizes.minTouchTarget). heightIn задаёт минимальную высоту 48.dp, а не жёсткую высоту. Это помогает сохранить удобный размер для нажатия. У IconButton стандартная область нажатия также около 48.dp.

> **Ответ: «Details получает id, находит объект в SampleData и показывает его поля. Прокрутка у Column. Кнопка меняет общий набор избранного через callback; её вид определяется isFavorite».**

14. Избранное, пустота и Preview

ui/screens/FavoritesScreen.kt → ui/components/EmptyState.kt

ui/screens/FavoritesScreen.kt | строки 39-39

```
val favorites = SampleData.offices.filter { it.id in favoriteIds }
```

Favorites выбирает записи, чьи id находятся в общем наборе. filter возвращает список ExchangeOffice. Это те же исходные объекты. Порядок сохраняется как в SampleData, а не в порядке добавления сердечек.

Если favorites.isEmpty(), показывается EmptyState. Иначе LazyColumn рисует OfficeCard. Здесь isFavorite = true: в список уже попали только избранные. Нажатие сердца удаляет id из общего набора; отбор пересчитывается. Удаление последней записи переключает экран на EmptyState.

ui/components/EmptyState.kt | строки 32-36

```
Column(
    modifier = modifier.padding(Spacing.extraLarge),
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.Center,
) {
```

EmptyState принимает icon, title, message и modifier. Это общий компонент для трёх случаев: нет обменников в городе, нет избранных, не найден id на Details. Различаются данные, а расположение одинаковое.

Родитель передаёт fillMaxSize, чтобы занять доступную область. Компонент добавляет padding 24.dp. CenterHorizontally центрирует детей поперёк Column; Arrangement.Center ставит их группу по вертикальному центру выделенной области. Без заполненной высоты большого свободного места для центрирования может не быть.

Внутри декоративный Icon 64.dp, затем заголовок и сообщение. contentDescription = null у значка: смысл уже объяснён текстом, отдельное повторение для чтения экрана не нужно. TextAlign.Center центрирует строки текста внутри Text; это не то же самое, что центрировать сам элемент в Column.

Что делают функции внизу файла

@OptIn(ExperimentalMaterial3Api::class) разрешает API Material 3, помеченное экспериментальным в этой версии. ::class передаёт ссылку на класс, а не создаёт объект. Это инструкция компилятору, не элемент UI.

Preview показывает компонент в Android Studio без полного запуска приложения. showBackground включает фон, UI_MODE_NIGHT_YES запрашивает тёмную конфигурацию. private ограничивает доступ функцией внутри файла. Пример передаёт пробные id и пустые callbacks {}, которые ничего не делают. Preview не является ещё одним маршрутом и сам по себе не доказывает работу нажатий.

15. Тема, размеры и ресурсы

ui/theme/ + res/ | общие настройки вместо значений в каждом экране

ui/theme/Theme.kt | строки 46-50

```
fun RateRadarTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable () -> Unit,
) {
    val colors = if (darkTheme) DarkColors else LightColors
```

darkTheme по умолчанию берётся из системной настройки. Boolean выбирает одну из двух схем. content: @Composable () -> Unit - переданная функция UI без аргументов. Далее MaterialTheme получает colorScheme, typography и content: оформление доступно всем вложенным вызовам.

Color.kt хранит Color(0xFF006A60) и другие значения. 0x означает шестнадцатеричную запись; FF - непрозрачность, следующие пары - красный, зелёный, синий. Theme.kt связывает эти значения с ролями: primary - акцент, surface - поверхность, onSurface - текст поверх неё.

Компонент читает MaterialTheme.colorScheme.surface, а не выбирает вручную светлый/тёмный цвет. Смена схемы даёт ему другую поверхность. Type.kt содержит Typography(): стандартные стили Material. titleMedium, bodySmall и headlineSmall - имена ролей текста, а не наши функции.

Dimens.kt хранит Spacing: 4, 8, 16, 24.dp. Sizes: 48.dp для минимального нажатия, 64.dp для пустой иконки, 180.dp для изображения. dp учитывает плотность экрана; размеры шрифтов используют sp, учитывающие ещё и настройку размера текста. Одна правка Spacing.large изменит все места, где его читают.

Почему R.string и R.drawable

Первые две строки - XML; следующие - обращения из Kotlin

```
<string name="label_city">City: %1$s</string>
<string name="rate_value">%1$.2f ₸</string>

stringResource(R.string.label_city, office.city)
painterResource(R.drawable.exchange_placeholder)
```

res/values/strings.xml хранит подписи. %1$s вставляет первый текстовый аргумент; %1$.2f - первый дробный с двумя знаками. R создаётся при сборке и содержит идентификаторы ресурсов. drawable содержит рисунки, mipmap - иконки запуска. XML внутри drawable описывает вектор через размеры и пути.

res/values/themes.xml и values-night/themes.xml задают тему Android-окна; Theme.kt задаёт тему UI Compose. Это разные уровни. Векторный placeholder локальный и одинаковый для всех обменников. Доступность задаём описаниями значков и изображений, размером нажатий и читаемыми цветами.

> **Меняем текст - strings.xml. Цвет - Color.kt и его роль в Theme.kt. Размер - Dimens.kt. Расположение элементов - соответствующая функция экрана или компонента.**

16. Android и сборка: что спросят рядом

AndroidManifest.xml + Gradle | не нужно учить версии наизусть

Android - система, Kotlin - язык, Jetpack Compose - библиотека UI. Material 3 даёт готовые панели, карточки, кнопки и тему. Android Studio - среда разработки. Gradle собирает исходники и ресурсы с библиотеками в приложение. SDK - набор инструментов и API Android.

AndroidManifest.xml сообщает системе об application и .MainActivity. intent-filter с MAIN и LAUNCHER делает Activity доступной как точку запуска из списка приложений. exported = true позволяет запуск извне, в том числе лаунчером. Три Compose-экрана отдельно в Manifest не регистрируются.

В корневом build.gradle.kts заданы версии плагинов Android и Kotlin. settings.gradle.kts подключает модуль app и репозитории библиотек. В app/build.gradle.kts находятся параметры приложения и dependencies. implementation подключает библиотеку для основного приложения; testImplementation - для тестов.

compileSdk = 36 - API, доступные при компиляции. minSdk = 24 - самая старая поддержанная версия API. targetSdk = 36 - версия поведения Android, на которую приложение рассчитано; это не запрет запускать на более новой системе. applicationId - идентификатор установленного приложения, namespace - пространство генерируемых R/BuildConfig.

compose = true включает поддержку Compose. BOM согласует версии Compose-библиотек; он не добавляет их все автоматически. navigation-compose подключает переходы; material3 - компоненты. gradlew запускает закреплённую проектом версию Gradle. APK - устанавливаемый пакет приложения.

Жизненный цикл и ограничения нашего решения

onCreate - создание; onStart - Activity видна; onResume - активна; onPause/onStop - теряет активность/видимость; onDestroy - уничтожение. При пересоздании начинается новая Activity. В проекте явно переопределён только onCreate. remember не переживает такое пересоздание, rememberSaveable использует сохранённое состояние для поддерживаемых значений. Диск и база - отдельная задача.

Нет ViewModel, Room, Retrofit и реального API. Нельзя сказать, что они здесь используются. Есть разделение данных, UI и навигации; одного этого недостаточно, чтобы называть проект полноценной Clean Architecture. ExampleUnitTest проверяет только 2 + 2, а не экраны.

Официальная документация для проверки понятий: состояние, списки, отступы Scaffold.

17. Вопросы: отвечай через свой код

Короткие ответы для ориентира; после ответа покажи соответствующий файл

«Что такое Scaffold?» Каркас экрана: у нас topBar с TopAppBar и основное содержимое. Он передаёт innerPadding; экран применяет его, чтобы содержимое не оказалось под панелью. Файл: ExchangeOfficeListScreen.

«Почему Row, Column и LazyColumn?» Row - дети рядом; Column - друг под другом; LazyColumn - прокручиваемый список с созданием элементов по необходимости. В OfficeCard Row делит информацию и сердце, Column складывает тексты.

«Как карточка знает, куда перейти?» Она не выбирает маршрут. Вызывает onClick. Экран подставляет office.id, а NavGraph получает id и выполняет navigate(details/id).

«Где хранится избранное?» В favoriteIds внутри RateRadarNavGraph. Это наблюдаемое множество id. toggleFavorite заменяет набор. Общий родитель передаёт новое значение всем экранам.

«Почему UI обновляется?» Во время построения читается MutableState. Изменение значения сообщает Compose о необходимости пересчитать зависящий UI. by лишь делает доступ короче; remember удерживает сам state между пересчётами.

«remember и rememberSaveable?» Первый держит значение в живом составе UI. Второй дополнительно умеет восстановление через saved state. У нас избранное - remember, город - rememberSaveable. После обычного нового запуска избранное пусто.

«Как работает фильтр?» selectedCity меняется по нажатию FilterChip. Для All берём все данные; иначе filter оставляет совпадающий city. Пустой результат даёт EmptyState. Исходные записи не удаляются.

«Зачем key?» Чтобы список узнавал карточку по постоянному id, даже когда её позиция меняется. Это не индекс, не маршрут и не ускоритель любой операции автоматически.

«Почему Back возвращает в Favorites?» Details был открыт поверх Favorites. popBackStack убирает последнее посещение. Новый navigate("offices") не равен возврату.

«Откуда фото и актуальные курсы?» Здесь локальная векторная иллюстрация и фиксированные демонстрационные числа в SampleData. Онлайн-обновления нет. Время updatedAt тоже текст.

«Почему тёмная тема меняет всё?» Theme.kt выбирает DarkColors по системному режиму, MaterialTheme передаёт роли вложенным компонентам. Компоненты читают эти роли вместо жёстких цветов.

«Что можно переиспользовать?» OfficeCard для списка и Favorites, RateBlock для карточки и Details, EmptyState для всех пустых случаев. Preview показывает примеры, но не является отдельным экраном приложения.

> **Если забыл название: сначала скажи, где лежат данные, какое событие произошло и какая функция получила событие. Затем найди эту цепочку в коде. Так легче объяснить смысл, чем вспоминать определение дословно.**

18. Последние 10 минут: проверь руками

Две небольшие правки + три сценария; исходный проект не изменён этим PDF

1. Измени видимый заголовок, 2 минуты

Открой app/src/main/res/values/strings.xml. В title_offices замени Exchange Offices на Exchange Offices KZ. Запусти приложение: меняется заголовок списка. Верни текст. Объяснение: Text читает R.string.title_offices; этот идентификатор связан со строкой XML.

2. Измени начальный фильтр, 3 минуты

Открой ui/screens/ExchangeOfficeListScreen.kt. В mutableStateOf(SampleData.ALL_CITIES) подставь "Astana". При свежем запуске ожидаются четыре карточки. Если восстановился старый город, сохранившееся rememberSaveable может перекрыть новый начальный вариант: для проверки удали приложение с эмулятора и снова Run. Верни ALL_CITIES.

3. Проследи события, 3 минуты

Список → открой id 1 → Add to favorites → Back → сердце заполнено → верхнее сердце → Favorites → открой запись → Remove → Back. Теперь пустое избранное. Проговори: callback → toggleFavorite → новый Set → пересчёт фильтра/кнопки. Затем выбери Karaganda: другой EmptyState, потому что нет данных города.

4. Без подсказки, 2 минуты

Покажи файлы для четырёх изменений: изменить курс; увеличить высоту картинки; добавить новый обменник; поменять текст пустого избранного. Ответы: SampleData.kt, Sizes.detailsImageHeight в Dimens.kt, снова SampleData.kt, empty_favorites_title/message в strings.xml. Новый обменник требует уникальный id; новый город нужно добавить ещё и в cities.

Если преподаватель попросит более сложную правку

Добавить поле phone: модель ExchangeOffice → аргументы всех записей в SampleData → Text в Details → новая подпись в strings.xml. Другой начальный экран: startDestination в NavGraph. Убрать многоточие: maxLines/overflow в OfficeCard. Поменять имя аргумента маршрута: шаблон DETAILS, navArgument и getInt одновременно.

> **Ты готов к основному разбору, когда можешь показать четыре цепочки: запуск → экран; город → отбор → список; карточка → id → Details; сердце → общий Set → Favorites. Если одна цепочка неясна, вернись к её странице, а не перечитывай документ целиком.**

Этот разбор относится к текущему RateRadarKZ в kbtu_android, а не к учебному android-practice. Он не обещает идеального выполнения каждого формального пункта SIS3: история коммитов, способ создания эскизов и сдача проверяются отдельно. Не приписывай проекту базу, API или постоянное сохранение, которых в нём нет.

19. State и public/private

Дополнение по вопросам на защите | сначала найди эти записи в проекте

State - данные, от которых зависит интерфейс

Две записи из разных файлов: NavGraph и экран списка

```
var favoriteIds by remember { mutableStateOf(setOf<Int>()) }
var selectedCity by rememberSaveable {
    mutableStateOf(SampleData.ALL_CITIES)
}
```

UI state - текущее состояние интерфейса. favoriteIds определяет сердца и список избранного, selectedCity - выбранный фильтр. isFavorite - переданный Boolean, вычисленный из набора, а не отдельное независимое хранилище. offices и favorites - списки, вычисляемые из данных и состояния.

mutableStateOf создаёт MutableState<T> с наблюдаемым value. При чтении Compose отслеживает зависимость; новое отличающееся значение запрашивает пересчёт нужного UI. remember держит объект между пересчётами. by делегирует чтение/запись, чтобы не писать .value. Без by: val state = remember { mutableStateOf(false) }; читать и менять нужно state.value.

Ещё state может означать режим экрана: Loading, Content, Empty, Error. Это обычные названия, не обязательные встроенные состояния Compose. У нас есть содержимое и EmptyState; загрузки из сети и её Loading/Error нет.

Recomposition - повторное выполнение нужных функций UI. Это не повторное создание Activity. rememberSaveable дополнительно сохраняет поддерживаемые значения для восстановления после пересоздания и системного завершения процесса, когда Android восстанавливает задачу. Это не постоянное хранение: закрытие задачи пользователем или обычный новый запуск не гарантируют восстановление.

Public/private - кто может обратиться к объявлению

Имена из проекта; ... обозначает опущенную часть

```
fun OfficeCard(...)              // public по умолчанию
private fun OfficeCardPreview()   // только этот файл
private val LightColors = ...    // только Theme.kt
```

public делает объявление доступным другим частям кода, которые видят его. Поэтому экран может вызвать OfficeCard из другого файла. private у объявления вне класса ограничивает доступ текущим файлом; private у члена класса - этим классом. Preview не нужен другим файлам, поэтому private. LightColors скрыта, а наружу доступна RateRadarTheme.

internal - доступ внутри одного модуля, здесь app. protected - член доступен в классе и его наследниках; для объявления вне класса не применяется. Эти слова управляют доступом в коде, а не сохранением данных и не безопасностью личной информации. Локальная favoriteIds внутри функции уже ограничена своей областью.

20. Lifecycle, Model и ViewModel

Дополнение по вопросам на защите | две разные темы, которые часто смешивают

Lifecycle - что Android делает с Activity

Типичные сценарии; это не обязательный путь для каждого события

```
Первое открытие: onCreate -> onStart -> onResume
Уход в фон:      onPause -> onStop
Возврат из фона: onRestart -> onStart -> onResume
Пересоздание:   старая Activity уничтожается,
                новая начинает с onCreate
```

onCreate - инициализация нового экземпляра; onStart - становится видимым; onResume - активен и готов к взаимодействию. onPause - теряет активное состояние, но может оставаться видимым; onStop - больше не видим. onDestroy - уничтожение экземпляра. При убийстве процесса системой onDestroy не гарантирован, поэтому на нём нельзя основывать обязательное сохранение.

Состояния Lifecycle: INITIALIZED, CREATED, STARTED, RESUMED, DESTROYED. После onPause Activity возвращается к STARTED, после onStop - к CREATED. Состояние и callback - разные вещи: callback сообщает о переходе.

У нас переопределён onCreate: super вызывает базовую реализацию, setContent подключает Compose. Поворот или смена конфигурации обычно пересоздаёт Activity. Избранное с remember тогда сбрасывается; город с rememberSaveable может восстановиться. Простая recomposition не вызывает эти методы.

Переход List → Details в нашем приложении меняет маршрут внутри той же MainActivity. Новая Activity для Details не создаётся. При этом у записи навигации есть собственный жизненный цикл; жизненный цикл composable связан с входом в Composition, пересчётами и выходом из неё. Это не отдельный onCreate для каждой функции экрана.

Model - данные. ViewModel - владелец состояния экрана

ViewModel у нас нет. Model - data class ExchangeOffice: поля одного обменника. SampleData - источник демонстрационных объектов. View/UI - Compose-функции экранов и компонентов. Папка model не означает, что в приложении уже есть ViewModel или вся архитектура MVVM.

Android ViewModel - отдельный класс для состояния и логики UI. Например, он мог бы хранить favoriteIds и selectedCity, обрабатывать toggleFavorite и отдавать значения экранам. Тогда экраны показывали бы данные и отправляли события. ViewModel сам не рисует Text и Card и не является базой данных.

Обычно ViewModel получают через Android API, которые привязывают экземпляр к владельцу: Activity, экрану навигации или графу. Тогда он переживает пересоздание этого владельца при изменении конфигурации. После окончательного удаления владельца очищается. Смерть процесса сам ViewModel не переживает: для восстановления используют SavedStateHandle, а для постоянного хранения - диск.

Проверка понятий: Kotlin visibility, ViewModel, Activity lifecycle.